



UNIVERSITY OF
MAURITIUS

MONTE CARLO SIMULATION FOR A SUPERMARKET

Name: MAKHUN Lokeshav Anand

Student ID: 2514204

Group: A1



BSc. (Hons) Computer Science
Module: ICT1207Y Computational Mathematics
Assignment Semester 2 (Level 1)
Lecturer Name: Dr. Pavaday Narainsamy

Table of Contents

Scenario	2
Part A – Theory	3
Explain what a Monte Carlo simulation is.	3
Why is simulation useful for queueing problems?	4
Why is the exponential distribution appropriate for arrival times?.....	5
Part B – Model Design	6
Random variables used	6
Probability distributions :.....	6
Simulation algorithm.....	7
Part C – Implementation	9
Libraries Used	9
Setting Parameters	13
Creating the Simulation Function.....	14
Displaying results for one simulation.....	16
Displaying results for 50 simulations	17
Creating Graphs	18
Part D – Repetition & Stability	20
Results Across 50 simulations Runs	20
Average waiting time and the standard deviation across simulations.	21
Comment on the variability of results.	21
Part E – Analysis & Recommendation	23
Do customers experience excessive waiting?	23
Should management add a third cashier?	24
Peak Hour Sensitivity Analysis:	25
Conclusion	25
References :	26

Scenario

A supermarket in Beau Bassin, Mauritius operates with 2 checkout counters. Management wants to estimate:

- Average customer waiting time
- Probability that a customer waits more than 5 minutes
- Whether a third cashier is needed

Because arrivals and service times are random, Monte Carlo simulation will be used.

Assumptions

- Customers arrive randomly at an average rate of 1 customer per minute
- Interarrival times follow an Exponential distribution
- Service time per customer is between 2 and 6 minutes, uniformly distributed
- The supermarket operates for 3 hours (180 minutes)

Part A – Theory

Explain what a Monte Carlo simulation is.

Monte Carlo simulation is a computational method that uses **repeated random sampling** to model the behaviour of a system involving uncertainty. Instead of solving a problem analytically, which is often impossible for complex real-world systems, we simulate the process many times, each time using different random inputs drawn from appropriate probability distributions and observe the statistical outcomes.

The method was developed during the 1940s and the simulation name originates from the **Monte Carlo Casino in Monaco**. It was primarily developed by mathematician Stanislaw Ulam, who was inspired by his uncle's gambling habits. Today it is widely used in finance, engineering, medicine and operations research.

For example, if a coin is repeatedly tossed and the outcomes are recorded, a simulation is being run. But a Monte Carlo simulation specifically uses a large number of random trials drawn from a known probability distribution to estimate statistical properties such as running 10,000 trials of tossing a coin and observing that heads appear approximately 50% of the time estimates the true probability.

In this assignment, Monte Carlo simulation models **customer arrivals** and **service times** at a supermarket checkout. Since we cannot predict exactly when each customer arrives or how long they take to serve, we generate these values randomly and observe the resulting waiting times over many simulations runs.

Why is simulation useful for queueing problems?

Queueing systems involve multiple interacting sources of randomness such as random arrivals, random service durations, and multiple servers. This makes them extremely difficult to analyse with closed-form formulas. While simple cases (such as M/M/1 queues) have known analytical solutions, more realistic systems quickly become intractable.

Simulation is valuable for queueing problems for several reasons:

- It handles any combination of distributions. In our case, service times follow a uniform distribution, which has no simple queueing formula.
- It captures time-dependent behaviour, such as arrival rates that change during peak hours, which analytical models struggle to represent.
- It allows us to test 'what if' scenarios cheaply without disrupting the real system, for example: comparing 2 versus 3 cashiers
- It produces directly actionable statistics: average waiting time, maximum wait, and probability of waiting more than 5 minutes.

The trade-off is that simulation results carry statistical uncertainty because they are based on random samples. This is why we repeat the simulation 50 times in Part D and analyse variability.

Why is the exponential distribution appropriate for arrival times?

The exponential distribution is the standard choice for modelling the time between random, independent events which is precisely what customer arrivals at a supermarket represent. It is appropriate for three reasons:

1. Independence

Each customer arrives independently. One person's decision to shop at a particular time has no effect on when the next customer arrives which is a condition the exponential distribution assumes directly.

2. Constant Average Rate

The exponential distribution is parameterised by rate λ (lambda), the average number of arrivals per unit time. With $\lambda = 1$, the average time between arrivals is $1/\lambda = 1$ minute. During peak hours, $\lambda = 1.5$ so the average gap shrinks to 40 seconds.

3. The Memoryless Property

This is the most important mathematical property. The probability of the next customer arriving within the next t minutes is the same regardless of how long you have already been waiting. Formally:

$$P(T > s + t \mid T > s) = P(T > t)$$

This property links directly to the Poisson process: if the number of arrivals in any time interval follows a Poisson distribution with rate λ , then the interarrival times follow an $\text{Exponential}(\lambda)$ distribution. To generate exponential random variables, we use the inverse transform method:

$$T = -(1/\lambda) \times \ln(U), \text{ where } U \sim \text{Uniform}(0, 1)$$

Part B – Model Design

Random variables used

Variable	Description
Interarrival Time (T)	Time in minutes between consecutive customer arrivals
Service Time (S)	Time taken in minutes for a cashier to service one customer

Note: Arrival time is computed as the cumulative sum of interarrival times. The simulation stops generating new customers once the arrival time exceeds 180 minutes. However, customers who arrived before the cutoff are still served - meaning departure times can exceed 180 minutes.

Probability distributions :

Variable	Distribution	Parameters	Mean	Formula
Interarrival Time	Exponential	$\lambda_{\text{normal}} = 1.0$ (normal) $\lambda_{\text{peak}} = 1.5$ (peak)	$\mu_{\text{normal}} = 1.0$ min $\mu_{\text{peak}} = 1.0$ min	$-(1/\lambda) \times \ln(U)$
Service Time	Uniform	a = 2 min b = 6 min	$\mu = 4.0$ min	$2 + 4U$

Simulation algorithm

Creating the Simulation Function

Step 1 — Initialisation

- Set simulation clock to 0.
- Set both cashiers as free: `cashier_free_at = [0, 0]`.
- Create an empty table to store customer records.

Step 2 — Generate Next Arrival

- Determine current λ : set $\lambda = 1.5$ if `current_time` ≥ 120 minutes (peak time), otherwise $\lambda = 1.0$. (normal time)
- Generate interarrival time $T = -(1/\lambda) \times \ln(U)$, where $U \sim \text{Uniform}(0,1)$.
- Update clock: `current_time = current_time + T`.
- If `current_time` ≥ 180 minutes, stop the simulation.

Step 3 — Assign to a Cashier

- Find the cashier who is free or who is free the soonest. (If both are free, then the first cashier is selected)
This is done by finding the smallest value in `cashier_free_at` (soonest free). The index at which it is found is the cashier which is free.
- Compute `service_start = max(arrival_time, cashier_free_at[i])`.
Service starts either at arrival time if the cashiers are free at that time or when the cashier is free the soonest.
- Compute `waiting_time = service_start - arrival_time`.
Waiting time means how long the customer must wait before checkout.

Step 4 — Generate Service Duration

- Generate service time $S = 2 + 4V$, where $V \sim \text{Uniform}(0,1)$. This gives a value uniformly distributed between 2 and 6 minutes.
- Compute `departure_time = service_start + S`.
Departure time means at what time the customer leaves the cashier after checking out.
- Update cashier availability: `cashier_free_at[i] = departure_time`.
Thus, this part is important for Step 3 to work correctly.

Step 5 — Record and Repeat

- Store all fields for this customer. This includes `interarrival_time`, `arrival_time`, `service_start`, `waiting_time`, `service_time`, `departure_time`, `cashier`, and `period`. Return to Step 2.

Step 6 — Summary Statistic

- When simulation clock exceeds 180 mins, the loop exits.
- Cashier utilisation = total service time / (number of cashiers × simulation duration). This measures the fraction of total available cashier time that was spent actively serving customers. A value above 100% means cashiers are always busy and the queue grows indefinitely.
- Compute average waiting time, maximum waiting time, and $P(\text{wait} > 5 \text{ min})$.

Note: this computation can be done outside the function using the pandas library.

Part C – Implementation

The implementation is divided into distinct parts, each responsible for a specific task in the simulation. Each part is presented below with its code, a description of what it does, and an explanation of every command used.

Libraries Used

Three external libraries are imported at the start of the program. A library is a collection of pre-written functions that extend what Python can do. Without these, generating random numbers, organising data into tables, and plotting graphs would require writing hundreds of lines of code from scratch.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

1.1) numpy (imported as np)

numpy (Numerical Python) is a library for fast mathematical operations, particularly on arrays of numbers. It is the foundation of the simulation because all random number generation depends on it.

Command Used	What It Does
np.random.exponential(1.0 / lam)	Generates a random interarrival time from the Exponential distribution with rate λ . Uses the formula $T = -(1/\lambda) \times \ln(U)$ internally
np.random.uniform(2, 6)	Generates a random service time uniformly distributed between 2 and 6 minutes. Every value in this range is equally likely
np.zeros(n_cashiers)	Creates an array of zeros — used to initialise cashier_free_at, meaning all cashiers start free at time 0
np.argmin(cashier_free_at)	Returns the index of the smallest value in the array — used to find which cashier becomes free soonest
np.sum(cashier_free_at > arrival_time)	Counts how many cashiers are still busy at the moment a customer arrives — gives the queue length
np.array(avg_waits)	Converts the list of average waiting times into a numpy array so statistical functions can be applied
avg_waits.mean()	Computes the mean (average) of all values in the array
avg_waits.std()	Computes the standard deviation — measures how spread out the values are around the mean
avg_waits.min() / avg_waits.max()	Returns the smallest / largest value in the array

1.2) pandas (imported as pd)

pandas is a library for organising, filtering, and analysing data in tabular form (rows and columns), similar to a spreadsheet. It stores all customer records from the simulation and makes it easy to compute statistics on specific subsets of data.

Command Used	What It Does
pd.DataFrame(records)	Converts the list of customer dictionaries into a table (DataFrame) where each row is a customer and each column is a field such as waiting_time or arrival_time
df['waiting_time'].mean()	Computes the average waiting time across all customers in the table
df['waiting_time'].max()	Finds the longest waiting time recorded in the simulation
(df['waiting_time'] > 5).mean()	Computes the proportion of customers who waited more than 5 minutes — this is $P(\text{wait} > 5 \text{ min})$
df['service_time'].sum()	Adds up all service times — used in the cashier utilisation formula
df[df['period'] == 'normal']	Filters the table to return only rows where the period column equals 'normal' — used to separate normal and peak results
df[df['period'] == 'peak']	Filters to peak period customers only
df.head(len(df)).to_string(index=False)	Prints all rows of the table as plain text without row index numbers

1.3) matplotlib.pyplot (imported as plt)

matplotlib is a library for creating graphs and charts. **pyplot** is its sub-module that provides a simple interface for building plots. It is used to produce the histogram of waiting times and the box and whisker plot.

Command Used	What It Does
plt.subplots(figsize=(w, h))	Creates a new figure and axes for the plot. <code>figsize</code> controls the width and height of the output image in inches
ax.hist(data, bins=20, ...)	Draws a histogram — divides the data into bins and counts how many values fall in each, showing the distribution of waiting times
ax.axvline(x, ...)	Draws a vertical line at a specific <code>x</code> value — used to mark the 5-minute threshold on the histogram
ax.axhline(y, ...)	Draws a horizontal line at a specific <code>y</code> value — used to mark the mean on the box plot
ax.boxplot(data, vert=False, whis=[0,100], ...)	Draws a horizontal box and whisker plot. <code>vert=False</code> makes it horizontal; <code>whis=[0,100]</code> stretches whiskers to the true min and max
ax.set_title(...)	Sets the title text displayed above the plot
ax.set_xlabel(...) / set_ylabel(...)	Labels the x-axis and y-axis
ax.legend()	Displays the legend showing what each colour or line represents
plt.tight_layout()	Automatically adjusts spacing so labels and titles are not cut off
plt.savefig('filename.png', dpi=150)	Saves the plot as a PNG image file at 150 dots per inch resolution
plt.show()	Displays the plot on screen

Setting Parameters

This part defines all the fixed values that control how the simulation behaves. These are written as named constants at the top of the program, so they are easy to find and change. For example, to simulate 5 cashiers instead of 2, only the N_CASHIERS line needs to be updated — nothing else in the code changes.

Code :

```
# — Parameters —
LAMBDA_NORMAL = 1.0      # customers per minute (normal period)
LAMBDA_PEAK    = 1.0      # customers per minute (peak period)
SIM_DURATION   = 180      # total minutes
PEAK_START     = 120      # peak begins at minute 120
N_CASHIERS     = 2
N_SIMULATIONS  = 50
```

Parameter	Purpose
LAMBDA_NORMAL = 1.0	Average customer arrival rate during normal hours → 1 customer per minute
LAMBDA_PEAK = 1.0*	Average arrival rate during peak hour → set to 1.2 or 1.5 depending on peak scenario
SIM_DURATION = 180	Total simulation time in minutes (3 hours)
PEAK_START = 120	The minute at which peak hour begins (last 60 mins = (180-60) = 120 mins)
N_CASHIERS = 2	Number of checkout counters (changing this to 3 for the 3-cashier scenario)
N_SIMULATIONS = 50	How many times the full simulation is repeated for Part D

Note: In the peak hour sensitivity analysis, LAMBDA_PEAK is changed to 1.2 and then 1.5 to compare how the system responds to different levels of demand increase.

Creating the Simulation Function

This is the core of the program. The `run_simulation()` function contains the entire logic of the Monte Carlo simulation. It generates customers one by one, assigns them to cashiers, computes their waiting times, and records everything in an array.

The function is written with default parameters so it can be called simply as `run_simulation()` for the standard scenario, or with custom values such as `run_simulation(n_cashiers=3)` to test different configurations without rewriting the logic.

The `while True` loop runs indefinitely until the simulation clock exceeds 180 minutes, at which point the `break` statement exits the loop. Each iteration of the loop represents one customer arriving.

Code :

```
# — Single simulation run —————
def run_simulation(n_cashiers=2, lambda_normal=1.0, lambda_peak=None,
peak_start=None):
    cashier_free_at = np.zeros(n_cashiers) # all cashiers free at start
    cust_id = 0 #customer ID
    records = [] # store each customer's data
    current_time = 0.0 # simulation clock

    while current_time < SIM_DURATION:
        # Determine current arrival rate
        if lambda_peak and current_time >= peak_start: #If there is a lambda peak and
the peak time has started
            lam = lambda_peak
        else:
            lam = lambda_normal

        # Generate next interarrival time (exponential)
        interarrival = np.random.exponential(1.0 / lam) #Exponential distribution of
interarrival time
        current_time += interarrival #Update current time

        if current_time >= SIM_DURATION: #If updated current time exceeds 180 min, the
program halts (the customer is not allowed to enter)
            break
```

```

    cust_id += 1
    arrival_time = current_time

    # Find the cashier who becomes free soonest --> returns the indices of the
    # minimum values
    cashier_idx = np.argmin(cashier_free_at)

    # Determine when service starts
    service_start = max(arrival_time, cashier_free_at[cashier_idx])
    # Time starts either when the customer arrives later than cashier is free, or
    # cashier is free after customer arrived

    # Waiting time
    waiting_time = service_start - arrival_time #waiting time is zero is
    service_start = arrival_time

    # Generate service duration (uniform between 2 and 6)
    service_time = np.random.uniform(2, 6) #uniform distribution

    # Departure time
    departure_time = service_start + service_time

    # Update cashier availability
    cashier_free_at[cashier_idx] = departure_time

    # Record this customer
    records.append({
        "customer_ID" : int(cust_id) ,
        "interarrival": round(interarrival,2),
        "arrival_time": round(arrival_time, 2),
        "service_start": round(service_start, 2),
        "waiting_time": round(waiting_time, 2),
        "service_time": round(service_time, 2),
        "departure_time": round(departure_time, 2),
        "cashier": cashier_idx + 1,
        "period": "peak" if (lambda_peak and arrival_time >= peak_start)
    else "normal"
    }) #here we use a dictionary

df = pd.DataFrame(records)

# Cashier utilisation: total busy time / (n_cashiers × sim duration)
total_busy = df["service_time"].sum()

utilisation = total_busy / (n_cashiers * SIM_DURATION)
return df, utilisation

```

Displaying results for one simulation

This part calls `run_simulation()` once and displays the results. It prints a summary of key statistics to the console, then prints the full customer-by-customer table so every arrival, wait, and departure can be inspected.

The DataFrame is split into two subsets — normal and peak — using pandas filtering (`df[df['period'] == 'normal']`). This allows the average waiting time to be compared separately for each period, which feeds directly into the peak hour sensitivity analysis.

Code :

```
#Output results and graphs
print("=== Single Run Results ===")
print(f"Customers served:           {len(df)}")
print(f"Average waiting time:       {df['waiting_time'].mean():.2f} min")
print(f"Maximum waiting time:        {df['waiting_time'].max():.2f} min")
print(f"P(wait > 5 min):             {(df['waiting_time'] > 5).mean():.2%}")
print(f"Cashier utilisation:          {utilisation:.2%}")
print()

# Normal vs Peak comparison
normal = df[df["period"] == "normal"]
peak   = df[df["period"] == "peak"]
print("=== Normal vs Peak Period ===")
print(f"Normal avg wait: {normal['waiting_time'].mean():.2f} min")
print(f"Peak avg wait:   {peak['waiting_time'].mean():.2f} min")
print()
```

Console Output :

```
=== Single Run Results ===
Customers served:           171
Average waiting time:       27.38 min
Maximum waiting time:        51.65 min
P(wait > 5 min):             89.47%
Cashier utilisation:          127.92%

=== Normal vs Peak Period ===
Normal avg wait: 48.95 min
Peak avg wait:   128.91 min
```

Displaying results for 50 simulations

This part repeats the full simulation 50 times using a for loop. Each run uses a different sequence of random numbers so each produces slightly different results. The average waiting time from each run is stored in a list called `avg_waits`.

After all 50 runs, the list is converted to a numpy array so that `.mean()`, `.std()`, `.min()`, and `.max()` can be applied. These four statistics describe the distribution of outcomes across all 50 runs and form the basis of the Part D variability analysis.

Code :

```
# — Part D: Repeat 50 times —————
avg_waits = []
for _ in range(N_SIMULATIONS):
    df_rep, _ = run_simulation(
        n_cashiers=N_CASHIERS,
        lambda_normal=LAMBDA_NORMAL,
        lambda_peak=LAMBDA_PEAK,
        peak_start=PEAK_START
    )
    avg_waits.append(df_rep["waiting_time"].mean())
avg_waits = np.array(avg_waits)

#for i, avg in enumerate(avg_waits, start=1):
#    print(f"Run {i:>2}: Average wait = {avg:.2f} min")

print()
print("=== 50 Simulation Repetitions ===")
print(f"Mean of avg waiting times: {avg_waits.mean():.2f} min")
print(f"Std dev of avg wait times: {avg_waits.std():.2f} min")
print(f"Maximum avg wait times: {avg_waits.max():.2f} min")
print(f"Minimum avg wait times: {avg_waits.min():.2f} min")
```

Console Output :

```
=== 50 Simulation Repetitions ===
Mean of avg waiting times: 87.31 min
Std dev of avg wait times: 14.01 min
Maximum avg wait times: 132.08 min
Minimum avg wait times: 57.66 min
```

Creating Graphs

This part produces two graphs using matplotlib: a histogram of customer waiting times from a single simulation run, and a box and whisker plot summarising the variability of average waiting times across all 50 runs.

Graph 1 — Histogram of Waiting Times

```
# Histogram of waiting times (single run)
avg_wait = df["waiting_time"].mean()
plt.figure(figsize=(10, 6))
plt.hist(df["waiting_time"], bins=20, color="#3498db", edgecolor="white", alpha=0.8,
label="Customer Count")
plt.axvline(5, color="#e74c3c", linestyle="--", linewidth=2, label="5 min threshold")
plt.axvline(avg_wait, color="#2ecc71", linestyle="-", linewidth=2, label=f"Average
Wait ({avg_wait:.2f} min)")
plt.grid(axis='y', linestyle=':', alpha=0.7)
plt.text(7, plt.gca().get_ylim()[1] * 0.9,
        f"CRITICAL:\n{((df[\"waiting_time\"] > 5).mean() * 100):.1f}% of
customers\nwait > 5 minutes",
        bbox=dict(facecolor='white', edgecolor='#e74c3c', boxstyle='round,pad=0.5'),
        color='#c0392b', fontweight='bold')
plt.title("Distribution of Customer Waiting Times" , fontsize=14, fontweight='bold',
pad=15)
plt.xlabel("Waiting Time (minutes)", fontsize=12)
plt.ylabel("Number of Customers", fontsize=12)
plt.legend(frameon=True, loc='upper right')
plt.tight_layout()
plt.savefig("waiting_times_histogram.png")
plt.show()
```

Graph 2 — Box and Whisker Plot

```
# Box and Whisker

fig, ax = plt.subplots(figsize=(8, 3)) # wide and short suits horizontal orientation

# Create the horizontal box and whisker plot
ax.boxplot(avg_waits,
            vert=False,                # horizontal orientation
            whis=[0, 100],            # whiskers stretch to true min and max
```

```

        patch_artist=True,          # allows the box to be filled with colour
        boxprops=dict(facecolor="#D6E4F0", color="#1F4E79"), # box fill and border
        medianprops=dict(color="#C00000", linewidth=2),      # median line
        whiskerprops=dict(color="#1F4E79", linewidth=1.5),    # whisker lines
        capprops=dict(color="#1F4E79", linewidth=1.5),        # caps at min/max
        flierprops=dict(marker=''))                             # no outlier dots

# Add a dashed vertical line for the mean
ax.axvline(np.mean(avg_waits),
           color="grey",
           linestyle="--",
           linewidth=1.5,
           label=f"Mean = {np.mean(avg_waits):.1f} min")

# Labels and formatting
ax.set_xlabel("Average Waiting Time (minutes)", fontsize=11)
ax.set_title("Variability of Average Waiting Time Across 50 Simulation Runs",
            fontsize=13, fontweight="bold")
ax.set_yticks([]) # hide y-axis tick – only one box so it's not needed
ax.legend(fontsize=10)
ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(False)
ax.spines["left"].set_visible(False)

plt.tight_layout()
plt.savefig("boxplot_variability.png", dpi=150)
plt.show()

```

Graph 3 — Normal vs Peak Waiting Time Comparison

```

# Normal vs Peak waiting time comparison
plt.figure(figsize=(8, 4))
plt.hist(normal["waiting_time"], bins=15, alpha=0.6, label="Normal",
color="steelblue")
plt.hist(peak["waiting_time"], bins=15, alpha=0.6, label="Peak", color="tomato")
plt.axvline(5, color="black", linestyle="--", label="5 min threshold")
plt.title("Waiting Times: Normal vs Peak Period")
plt.xlabel("Waiting Time (minutes)")
plt.ylabel("Number of Customers")
plt.legend()
plt.tight_layout()
plt.savefig("normal_vs_peak.png")
plt.show()

```

Part D – Repetition & Stability

Results Across 50 simulations Runs

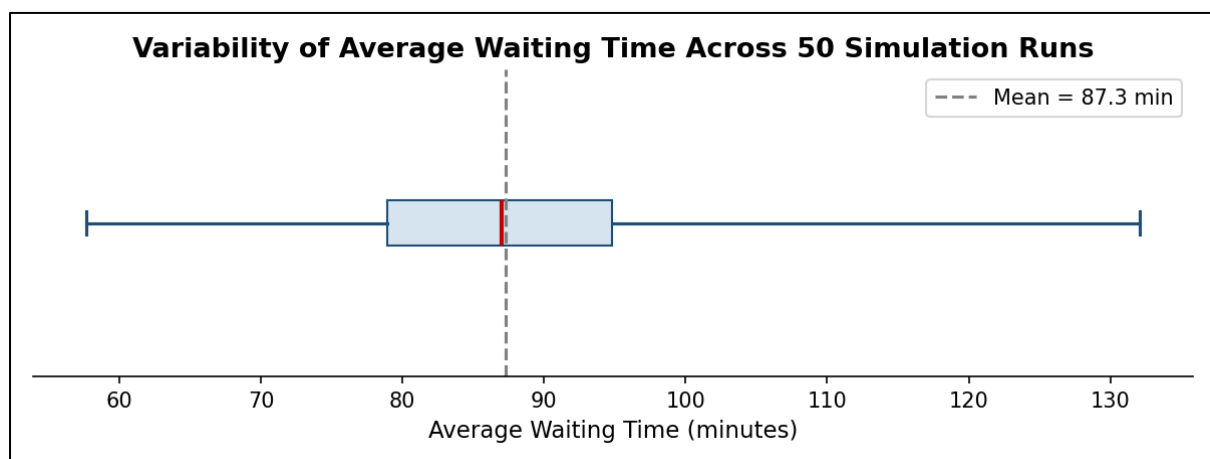
The full simulation was repeated 50 times. The average waiting time was recorded for each run.

Run #	Average Wait (min)	Run #	Average Wait (min)
Run 1	82.63	Run 26	78.65
Run 2	66.89	Run 27	67.19
Run 3	97.43	Run 28	82.72
Run 4	87.71	Run 29	68.14
Run 5	94.94	Run 30	86.31
Run 6	132.08	Run 31	76.05
Run 7	88.53	Run 32	101.09
Run 8	97.33	Run 33	98.47
Run 9	81.41	Run 34	88.63
Run 10	97.24	Run 35	81.61
Run 11	73.97	Run 36	94.34
Run 12	119.67	Run 37	99.08
Run 13	102.96	Run 38	79.74
Run 14	93.56	Run 39	84.15
Run 15	81.11	Run 40	93.68
Run 16	84.96	Run 41	70.77
Run 17	85.53	Run 42	70.86
Run 18	107.35	Run 43	88.58
Run 19	112.00	Run 44	94.22
Run 20	68.00	Run 45	78.40
Run 21	89.58	Run 46	78.53
Run 22	99.08	Run 47	91.21
Run 23	65.18	Run 48	89.94
Run 24	82.57	Run 49	57.66
Run 25	80.37	Run 50	93.60

Average waiting time and the standard deviation across simulations.

Statistic	Value
Mean of average waiting times (50 runs)	87.31 minutes
Standard deviation	14.01 minutes
Minimum average wait (best run)	57.66 minutes
Maximum average wait (worst run)	132.08 minutes

Comment on the variability of results.



1. The Spread (Whiskers)

The whiskers extend from the minimum (57.66 min) to the maximum (132.08 min), giving a total range of approximately 74 minutes. This wide range indicates that the simulation results are highly sensitive to the random sequence of arrivals and service times generated in each run.

2. The Box (Interquartile Range)

The box spans from approximately 79 to 95 minutes (IQR $\approx$ 16 minutes), meaning the middle 50% of simulation runs produced average waiting times within this 16-minute band. While this is narrower than the full range, a 16-minute IQR is still significant which shows that even 'typical' runs vary considerably.

3. Skewness

The median (~86 min) sits slightly below the mean (87.31 min), indicating a mild positive skew. This is characteristic of queueing systems: the distribution cannot go below zero, but extreme clustering of arrivals can push individual runs to very high average waits, pulling the mean above the median.

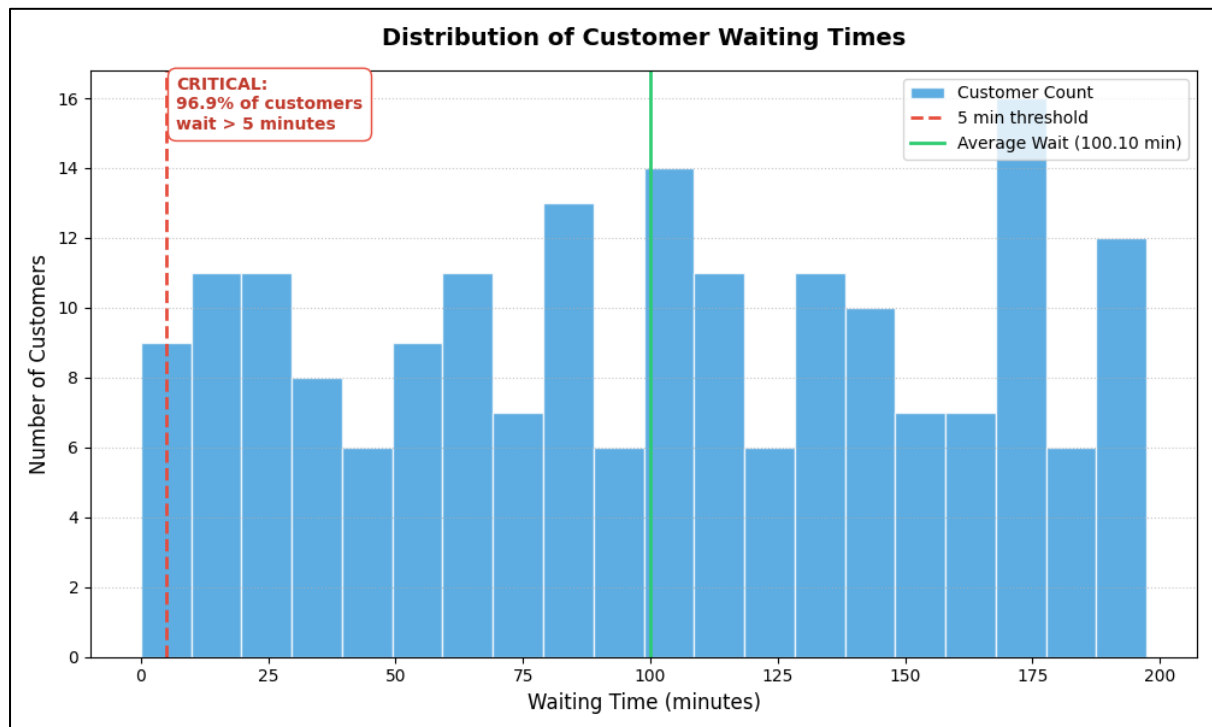
Summary

The high variability across 50 runs confirms that the two-cashier system is not just occasionally overloaded - it is structurally unstable on every run. The best case (57.66 min average) still represents completely unacceptable waiting times, demonstrating that the problem cannot be attributed to random chance.

Part E – Analysis & Recommendation

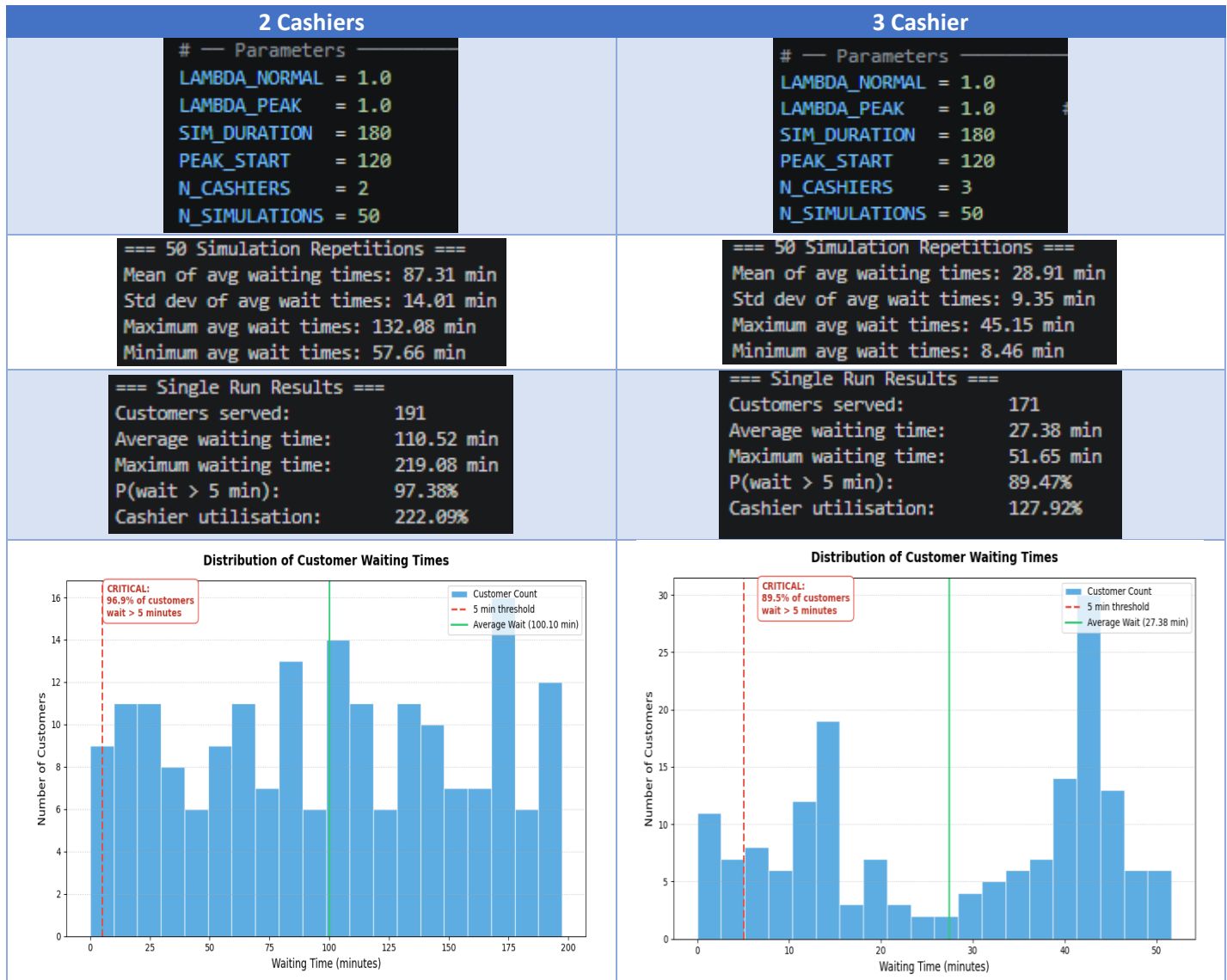
Do customers experience excessive waiting?

To determine whether customers experience excessive waiting, the simulation records each customer's waiting time and produces a histogram showing the distribution across all customers in one run:



From the graph, where the red line shows the 5 minutes mark, 96.9% of customers wait more than 5 minutes, while the average wait (green line) is around 100 minutes. This is a very large waiting time which shows a fundamental capacity failure that makes shopping in the supermarket entirely impractical. Thus, it can be concluded that customers experience excessive waiting.

Should management add a third cashier?



A noticeable difference in average waiting time and cashier utilization can be seen by increasing the number of cashiers to 3. A decrease of 58.40 minutes (roughly about 65 % decrease) in average waiting time is a major improvement. However, a third cashier alone will not resolve the bottle-necking problem.

The cashier utilization remains above 100%, which implies not only that cashiers are still always busy and the queue continues to grow throughout the simulation duration .

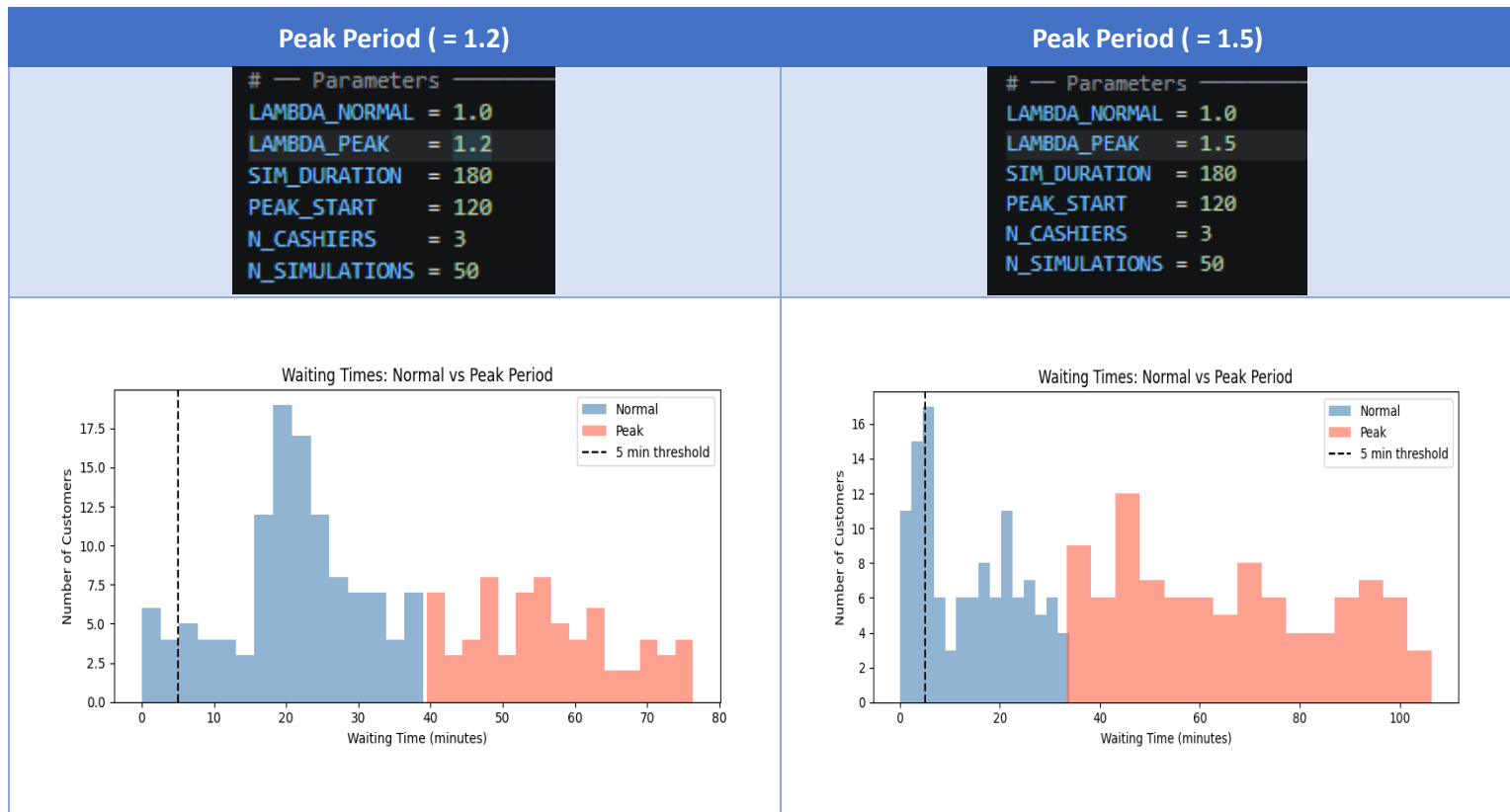
Using the queuing theory, the traffic intensity with 3 cashiers would still be $\rho = 1.0 \times 4.0 / 3 \approx 1.33$ — still above 1, meaning the queue would still grow indefinitely. Theoretical stability requires at least 5 cashiers ($\rho = 1.0 \times 4.0 / 5 = 0.80$).

Peak Hour Sensitivity Analysis:

Modify the model so that:

- Arrival rate increases during peak hour (last 60 minutes)
- Compare performance before and during peak

Comment on how sensitive the system is to small increases in arrival rate.



Comment : The system is **extremely sensitive** to changes in the arrival rate. A 25% increase in peak demand (from 1.2 to 1.5) resulted in a disproportionate increase in wait times, with the maximum wait jumping from ~75 to over 100 minutes. This indicates that at $\lambda = 1.5$, the 3-cashier setup has reached its 'saturation point,' where the arrival rate exceeds the service capacity, leading to an unstable and rapidly growing queue.

Conclusion

The Monte Carlo simulation demonstrates that 2 cashiers are entirely insufficient for the given parameters, and adding one more cashier will not resolve the overflow problem. As stated above, at least 5 cashiers will be needed for stability.

References :

- I. Wikipedia Contributors (2019). Monte Carlo method. [online] Wikipedia. Available at: https://en.wikipedia.org/wiki/Monte_Carlo_method.
- II. Cook, J.D. (2008). What happens when you add a new teller? Available at: <https://www.johndcook.com/blog/2008/10/21/what-happens-when-you-add-a-new-teller/>
- III. Cook, J.D. (2022). M/M/2 queue. Available at: <https://www.johndcook.com/blog/2022/01/12/mm2/>
- IV. Ledder, G. (n.d.). Queueing Theory Notes. University of Nebraska. Available at: <https://www.math.unl.edu/~gledder1/Notes/428/Queueing%20Theory%20Notes.pdf>
- V. GeeksforGeeks. "Introduction to Matplotlib." *GeeksforGeeks*, 14 May 2018, www.geeksforgeeks.org/python/python-introduction-matplotlib/.
- VI. W3Schools. "Introduction to NumPy." *Www.w3schools.com*, 2022, www.w3schools.com/python/numpy/numpy_intro.asp.
- VII. "Pandas Tutorial." *Www.w3schools.com*, 2024, www.w3schools.com/python/pandas/default.asp.
- VIII. Yi, Mike. "A Complete Guide to Box Plots." *Atlassian*, 2025, www.atlassian.com/data/charts/box-plot-complete-guide.